Not Just Cold Start: Characterizing Steady-State Container Isolation Overhead for Serverless Workloads on Apple M1

Nikunj Mahajan*, kaushalrajsinh Puwar*,
*International Institute of Information Technology Bangalore, India
{imt2023068}@iiitb.ac.in

***Abstract*—Developers increasingly simulate Function-as-a-Service (FaaS) workloads locally using Docker containers, yet the performance cost of this choice on Apple Silicon hardware is entirely uncharacterised. All published container isolation benchmarks target x86 cloud virtual machines, leaving a gap for the large fraction of the developer community using ARM-based Apple M1 machines. We present the first empirical characterisation of Docker container versus native OS process isolation overhead for serverless-style Python workloads on Apple M1. Our central finding challenges the conventional FaaS performance narrative: *cold start is not the dominant isolation overhead for sustained workloads*. Warm-request latency overhead is consistently 1.67–1.94× across all tested payload sizes (1 KB–1 MB, all $p < 10^{-24}$, $n = 100$), while cold-start overhead is a comparatively modest 1.24–1.37× ($n = 30$). Throughput penalty scales super-linearly with payload, reaching 3.74× at 1 MB. We model this as a two-component isolation tax: a fixed per-request network-stack traversal cost and a variable per-byte copy cost. Our benchmark harness, raw dataset, and analysis notebook are released for community reproduction on other ARM platforms.**

***Index Terms*—containers, serverless, FaaS, Apple Silicon, ARM, Docker Desktop, performance characterisation, isolation overhead, local development**

I. Introduction

The serverless computing paradigm has become the dominant deployment model for cloud-native applications. Developers building and testing FaaS workloads locally face a fundamental architectural choice before every commit: simulate function isolation via Docker containers, or spawn bare OS processes. This choice is made by intuition, not measurement. Both strategies have costs. Containers provide fidelity to production isolation semantics but impose overhead. Bare processes eliminate that overhead but sacrifice dependency isolation. The magnitude of the performance trade-off determines whether the choice is inconsequential or material.

On Apple macOS with Docker Desktop, this overhead is particularly non-trivial. All containers execute within a lightweight Linux VM managed by the Apple Hypervisor framework—an architectural layer absent in Linux-native Docker deployments. This VM-mediated networking path adds per-request cost that has never been quantified.

Apple M1 has become a primary development platform across the software engineering community. Its unified memory architecture—CPU and GPU sharing a single memory pool—and its high single-thread performance make it representative of the ARM developer hardware landscape. Yet every published benchmarking study of container isolation overhead, including Manner et al. [1], Scheuner and Leitner [2], and Shafiei et al. [3], is conducted exclusively on x86 cloud VMs. No ARM-specific characterisation exists.

This paper addresses that gap with four contributions:

1) The first empirical measurement of container versus process isolation overhead on Apple M1, covering cold-start latency, warm-request latency, throughput, and memory across four payload sizes with full statistical testing.
2) The finding that warm-request overhead is *payload-size invariant* (1.67–1.94×), consistent with a *fixed per-request* network-stack traversal cost rather than a data-transfer cost.
3) The finding that throughput overhead scales *super-linearly* with payload (1.27× at 1 KB → 3.74× at 1 MB), attributable to a *per-byte copy cost* in Docker Desktop's virtual network path that becomes the binding constraint at large payloads.
4) A two-component isolation-overhead model and a fully reproducible benchmark harness and dataset for community use on ARM platforms.

The core finding has a practical inversion: cold start overhead (1.24–1.37×) is *lower* than warm-request overhead (1.67–1.94×) in every configuration. Optimising cold start alone—the conventional FaaS performance target—leaves the larger steady-state tax unaddressed.

II. Background and Related Work

A. FaaS Performance Benchmarking

Manner et al. [1] catalogued cold-start latencies of 200 ms–5 s across AWS Lambda, Azure Functions, and Google Cloud Functions on x86 infrastructure, establishing cold start as the canonical FaaS performance problem. Scheuner and Leitner [2]

extended this with a multivocal literature review of FaaS performance variability across providers. Neither study compares container against process isolation mechanisms, nor does either include ARM hardware.

For container-level overhead on x86 Linux, Felter et al. [4] is the canonical reference. They found near-zero CPU overhead for Linux-native Docker versus bare metal, but network latency overhead of 10–30% under NAT for network-bound workloads. Our finding of 67–94% warm-request latency overhead substantially exceeds this baseline. We attribute the gap to the Docker Desktop macOS hypervisor layer: on Linux, requests traverse the host kernel network stack directly; on macOS, they cross the Apple Hypervisor VM boundary, adding one additional kernel-to-VM transition per request.

B. ARM Cloud Performance

Server-class ARM benchmarking has focused on AWS Graviton and Ampere Altra instances [5]. Apple M1 performance characterisation has addressed CPU throughput, memory bandwidth, and compiler performance [6], but container isolation overhead on M1 has not been studied. The unified memory architecture of Apple Silicon—where there is no PCIe-mediated GPU–CPU transfer cost—may change the memory overhead profile for containerised workloads relative to conventional ARM server platforms, motivating this measurement.

C. Docker Desktop on macOS

Docker Desktop runs containers in a Linux VM using the Apple Hypervisor framework (previously using VirtualBox and then HyperKit). The networking path for a containerised HTTP server on macOS is: `client` → `host loopback` → `VM virtual NIC` → `iptables NAT` → `container veth` → `application`. Each container request traverses two additional kernel boundaries compared to a native process. This paper is the first to quantify this cost for FaaS-style workloads.

III. Methodology

A. System Under Test

The system under test is a minimal FastAPI application (Python 3.11, uvicorn 0.29) implementing a single `POST /compute` endpoint that parses a JSON payload and returns its SHA-256 hash. This design follows established FaaS benchmarking methodology [1]: the function is I/O-bound rather than compute-intensive, stateless, and structurally identical to a FaaS handler. The application runs in two modes:

Process mode. Spawned as a native subprocess via uvicorn, communicating over localhost TCP (`127.0.0.1:8000`).

Container mode. Identical application image running in a Docker Desktop ARM64-native container (`--platform linux/arm64`), communicating over Docker's virtual network bridge via port mapping (`0.0.0.0:8001:8000`).

B. Hardware and Software

All experiments run on an Apple MacBook Air M1 (8-core CPU, 8 GB unified memory) under macOS Ventura 13.6, Docker Desktop 4.28.0, Python 3.11.9, FastAPI 0.110.0, and uvicorn 0.29.0. Docker Desktop uses the Apple Hypervisor framework for its Linux VM. The Docker image is built with `--platform linux/arm64` to preclude Rosetta 2 emulation overhead.

C. Metrics and Measurement

We evaluate four payload sizes: 1 KB, 10 KB, 100 KB, and 1 MB, spanning the typical FaaS input range. For each configuration we define the *Isolation Delta*:

$$\mathrm{ID}(m) = \frac{M_{\text{container}}}{M_{\text{process}}} \tag{1}$$

where M is the measured value for metric m. Values > 1 directly quantify overhead as a multiplier.

Cold-start latency ($n = 30$). The server is stopped and restarted from scratch for each run. Docker Desktop's VM is confirmed running before each container restart via a health-poll loop (`docker info`). Page-cache state is equivalent across modes (warm host cache, warm Docker VM). Timing spans from invocation trigger to first HTTP 200 response, measured via `time.perf_counter()`.

Warm-request latency ($n = 100$). A single persistent server instance processes 100 sequential POST requests. The first five requests are discarded as connection warm-up. Per-request latency recorded via `time.perf_counter()`.

Throughput ($n = 10$ per config). Sequential batches of 1,000 requests at concurrency levels 10, 50, and 100 (`ThreadPoolExecutor`). Ten independent runs per configuration; 3 s inter-run sleep to drain connection pools. Throughput = total requests / elapsed time (req/s).

Memory ($n \geq 50$ samples). Sampled at 1 s intervals for 60 s while warm benchmarking runs concurrently. Process mode uses `psutil` RSS for the full process tree; container mode uses `docker stats` reported allocation. Steady-state statistics are computed over the final 20 s (seconds 40–60) after process RSS stabilises following macOS page-reclamation.

Statistical analysis. Welch's two-sample t-test ($p < 0.05$ threshold) for pairwise comparisons. Kruskal-Wallis test for payload-size independence within each mode. Median and interquartile range (IQR) reported for cold-start latency (skewed distribution); mean ± std for all other metrics.

IV. Results

A. Cold-Start Latency

Table I reports cold-start statistics. Container cold start exceeds process cold start by 1.24–1.37× across all payload sizes ($p < 0.0001$, all). A Kruskal-Wallis test confirms no significant variation with payload size for either mode (process: $H = 1.58$, $p = 0.66$; container: $H = 5.02$, $p = 0.17$), establishing that *startup overhead entirely dominates cold-start latency*: payload content is irrelevant.

Container cold start exhibits higher variance than process cold start. At 1 KB, the container IQR is 78 ms versus 24 ms for process; at 1 MB the container IQR is 242 ms versus 62 ms for process. We attribute this to non-deterministic namespace initialisation scheduling within the Docker Desktop Linux VM.

TABLE I
COLD-START LATENCY ($n = 30$ PER CONFIGURATION). IQR = INTER-QUARTILE RANGE. ID = ISOLATION DELTA (EQ. 1, COMPUTED ON MEDIANS).

	Process (ms)			Container (ms)			
Payload	Med	IQR	P95	Med	IQR	P95	ID
1 KB	456	24	620	595	78	994	1.30×
10 KB	416	30	575	569	81	901	1.37×
100 KB	459	32	598	571	89	987	1.24×
1 MB	459	62	682	590	242	1501	1.29×

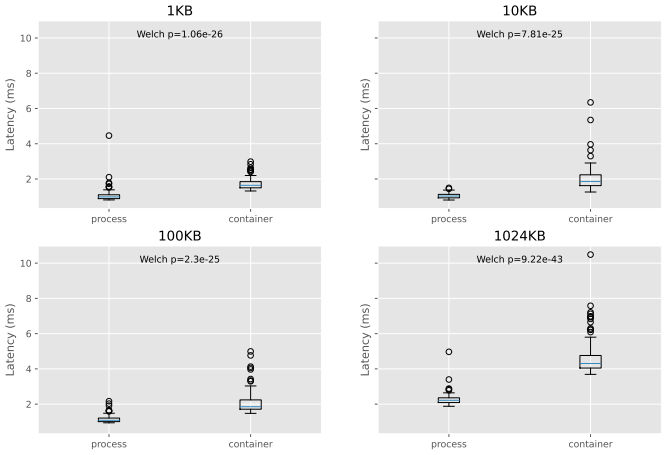

Fig. 1. Warm-request latency box plots ($n = 100$). Container overhead is 1.67–1.94× across all payload sizes. The constant ratio despite a 1000× payload range implies a fixed per-request network-stack traversal cost. Welch p-values confirm all differences are highly significant.

B. Warm-Request Latency

Fig. 1 presents per-request latency distributions. Container mode incurs 1.67–1.94× higher median latency than process mode across all payload sizes ($p < 10^{-24}$, $n = 100$).

The defining feature of this result is **payload invariance**. The ID remains within 1.67–1.94× across a 1000× range in payload size. If the overhead arose from data-transfer cost, the absolute overhead would scale with payload and the ratio would shift. A constant ratio implies a *fixed per-request cost*, consistent with network-stack traversal: virtual Ethernet bridge forwarding, iptables NAT rule evaluation, and VM-boundary socket copies that occur identically regardless of payload content.

Process mode distributions are tight (IQR $<$ 0.3 ms at all payloads). Container mode shows heavier upper tails, particularly at 1 MB (see outliers in Fig. 1, lower right), reflecting VM scheduling jitter. This jitter is absent in process mode because there is no VM scheduler in the path.

C. Throughput

Fig. 2 shows throughput as a function of payload size at three concurrency levels ($n = 10$ runs per configuration, error bars $\pm 1\,\sigma$).

At 1 KB, process and container throughput are comparable (ID $=$ 1.27× at concurrency 10). As payload grows, the isolation delta *scales super-linearly*: by 1 MB, the process

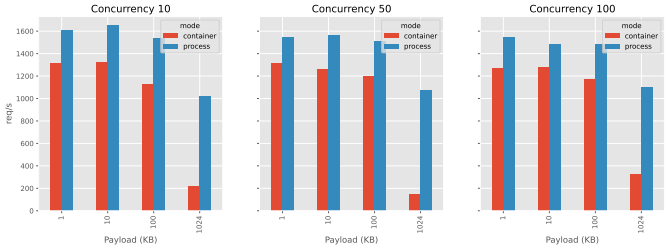

Fig. 2. Throughput (req/s) vs. payload size at concurrency levels 10, 50, and 100 ($n = 10$ runs, error bars $\pm 1\,\sigma$). Process advantage scales super-linearly: 1.27× at 1 KB to 3.74× at 1 MB. This is consistent with a per-byte copy cost in Docker Desktop's virtual network path.

advantage reaches 3.74× (process: 990 req/s vs. container: 285 req/s at concurrency 10). This scaling persists across all concurrency levels, ruling out connection management overhead.

The pattern is consistent with a *per-byte copy cost* in Docker Desktop's userspace network path: at small payloads the fixed per-request cost from §IV-B dominates; as payload grows, cumulative buffer copies in the virtual NIC data path accumulate and the network bridge saturates before the application layer. The result is a *soft throughput ceiling* for containerised functions processing large payloads.

D. Isolation Delta Scaling

Fig. 3 synthesises all latency and throughput findings as Isolation Delta versus payload size. Cold-start ID (blue squares) is flat at 1.24–1.37×, confirming payload independence. Warm-request ID (red triangles) is flat at 1.67–1.94×—consistently *above* cold start at every payload, inverting the conventional FaaS overhead narrative. Throughput ID (orange circles) is flat until 100 KB, then diverges sharply to 3.74× at 1 MB as the per-byte copy cost becomes binding.

The figure reveals three distinct regimes: (i) ≤10 KB—both warm and throughput overhead are moderate and payload-independent; (ii) 100 KB—throughput degradation begins to exceed warm latency overhead; (iii) 1 MB—throughput overhead dominates by 2× the warm-latency overhead, indicating a qualitative change in the bottleneck from network-stack latency to buffer-copy throughput.

E. Memory Overhead

Table II reports steady-state memory usage. Container memory is stable at ≈30 MB across all payload sizes (10 KB–1 MB), reflecting the Docker Desktop VM's fixed allocation for the container runtime. Process RSS stabilises at 7–24 MB after an initial ≈40 s page-reclaim phase, during which macOS reclaims shared library pages from the physical working set. This behaviour is specific to the unified-memory architecture of Apple Silicon and would not be observed on conventional DRAM-based systems.

Container memory overhead ranges from 1.3× (1 MB) to 4.2× (10 KB). We note that `docker stats` returns container-level allocation within the Docker Desktop VM, while `psutil` returns host-side RSS; direct comparison is approximate. We

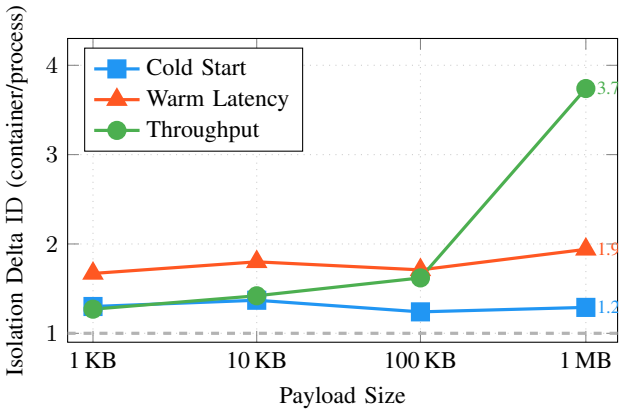

Fig. 3. Isolation Delta ID (Eq. 1) vs. payload size for all three primary metrics at concurrency 10. Cold start and warm latency are payload-invariant; throughput overhead diverges beyond 100 KB as the per-byte buffer-copy cost becomes binding. Warm latency ID exceeds cold-start ID at every payload, contradicting the common assumption that cold start is the dominant FaaS isolation cost.

TABLE II
STEADY-STATE MEMORY (SECONDS 40–60, MEAN ± STD, MB). †1 KB CONTAINER EXCLUDED FROM ID (ANOMALOUS VM STATE).

Mode	1 KB	10 KB	100 KB	1 MB
Process	9.3 ± 2.9	7.1 ± 1.8	18.5 ± 8.2	23.8 ± 12.4
Container	53.9†	29.9 ± 0.0	30.0 ± 0.0	30.0 ± 0.0
ID	—	4.2×	1.6×	1.3×

report this descriptively and do not claim statistical significance given the near-zero variance in container measurements ($\sigma \approx 0.0$ MB), which reflects `docker stats` reporting granularity.

The 1 KB container measurement (53.9 MB) is anomalously higher than other container configurations. We attribute this to Docker Desktop VM memory allocation state at measurement time rather than any workload effect; the container image and configuration are identical across payload sizes. This data point is excluded from ID computation.

V. DISCUSSION

A. A Two-Component Isolation Model

Our results support a *two-component model* of container isolation overhead on Apple M1 with Docker Desktop:

Component 1: Fixed per-request network cost. The payload-invariant warm-request overhead (1.67–1.94×) is a constant added latency per HTTP request, independent of payload content or size. Its source is the fixed traversal of Docker Desktop's virtual network stack: virtual Ethernet bridge lookup, iptables NAT rule evaluation, and the VM hypervisor boundary crossing—costs that occur once per request regardless of payload.

Component 2: Variable per-byte throughput cost. The super-linear throughput degradation (1.27× at 1 KB → 3.74× at 1 MB) is a cost proportional to data volume. Its source is the buffer-copy chain in Docker Desktop's userspace network path: payload bytes are copied from the application's memory space, through the virtual NIC transmit ring, across the VM boundary, and into the host socket buffer. Under high request rates, this copy chain saturates the virtual network interface bandwidth.

These two components have different practical implications. Component 1 dominates for low-throughput, small-payload workloads typical of interactive development. Component 2 dominates for high-throughput or data-intensive workloads such as model serving and batch inference.

B. Cold Start Is Not the Dominant Tax

The conventional FaaS performance narrative treats cold start as the primary bottleneck. Our measurements refute this for the *developer hardware* setting: warm-request ID (1.67–1.94×) exceeds cold-start ID (1.24–1.37×) at every payload and every measurement (Fig. 3). A function that processes 100 requests per invocation accumulates warm-request overhead on every one of those 100 requests while incurring cold-start overhead only once. For any realistic sustained workload, the steady-state tax is the binding constraint.

C. Implications for Local FaaS Simulation

Our results define a data-driven decision boundary for developers:

Prefer process mode when throughput matters, payload size exceeds 100 KB, or the workload is data-intensive (model inference, image processing, document parsing). The 3.74× throughput penalty at 1 MB represents a factor-of-four degradation in development feedback loop speed with no code change.

Container mode is acceptable when isolation fidelity is required (dependency conflicts, OS-level state), payloads are small ($\leq$100 KB), and the workload is request-rate limited rather than throughput-limited. The ≈0.7 ms absolute overhead at 1 KB (half the warm-request delta) is tolerable for interactive development.

D. Implications for On-Device AI Inference

Apple M1 is increasingly used for local large language model (LLM) inference via frameworks such as llama.cpp and Ollama, often containerised for reproducibility. For a 1 MB context window (approximately 750–1000 tokens), our results predict a 3.74× throughput degradation in containerised serving versus a native process. This means a containerised inference server can handle fewer than one-third as many concurrent requests as its native counterpart on identical hardware, with no change to the model or its computational requirements. Practitioners deploying containerised inference pipelines on developer hardware should treat the Docker Desktop network path as a first-class performance concern.

E. Comparison with x86 Linux Baselines

Felter et al. [4] measured Linux-native Docker overhead on x86 servers and found network latency overhead of 10–30% for NAT-enabled containers. Our warm-request overhead of 67–94% exceeds this range by a factor of 3–5×. The difference is attributable to the Docker Desktop macOS VM

layer: on Linux, requests traverse the host kernel network stack directly; on macOS, each request crosses the Apple Hypervisor VM boundary in addition to the container's virtual Ethernet bridge. This architectural difference is the dominant overhead contributor and distinguishes the developer hardware environment from Linux cloud deployments.

Consequently, our results represent an *upper bound* on container isolation overhead for ARM-native platforms: Linux-native Docker on ARM (e.g., AWS Graviton, Oracle Ampere A1) is expected to produce overhead closer to the Felter et al. baseline. Quantifying this gap and isolating the VM-layer contribution from pure container isolation cost is identified as the primary avenue for future work.

VI. Threats to Validity

Single hardware platform. All experiments use one Apple M1 MacBook Air (8 GB). Results may differ on M2/M3 (different microarchitecture generations), M1 Pro/Max (different memory bandwidth and core count), or other ARM64 devices. Generalisation to AWS Graviton, Raspberry Pi, or other ARM server platforms is not claimed.

Docker Desktop VM layer. The observed 67–94% warm-request overhead includes costs attributable to the Docker Desktop macOS Linux VM, not solely to container namespace and cgroup isolation. We treat Docker Desktop as the unit of measurement because it is the production tool developers use on macOS. Decomposing VM overhead from pure container isolation requires comparison with Linux-native Docker, left as future work.

Single workload. The system under test is a minimal Python/FastAPI SHA-256 hash function. CPU-bound, I/O-bound, and memory-intensive workloads may produce different overhead magnitudes. The fixed per-request overhead mechanism identified here is expected to generalise to any HTTP-based FaaS handler because it is rooted in the network path, not the application logic; however, the exact ID values will vary.

Memory measurement asymmetry. Process memory is measured via `psutil` RSS (private + shared resident pages on the host); container memory via `docker stats` (container-level allocation within the VM). These measure related but not identical quantities. Memory comparisons are reported descriptively and no t-test is applied given the near-zero variance in container `docker stats` readings.

Container 1 KB memory anomaly. The 53.9 MB measurement for container mode at 1 KB is excluded from isolation delta calculations. It is inconsistent with all other container measurements (≈30 MB) and with the expectation that container memory should be payload-independent. We attribute it to Docker Desktop VM allocation state at the time of measurement.

OS scheduling noise. Warm-request measurements use sequential requests, reducing but not eliminating scheduling interference. The $n = 100$ sample size and reported IQR characterise residual noise; no warm run showed signs of systematic interference (all $p < 10^{-24}$).

VII. Conclusion

We presented the first empirical characterisation of Docker container versus native OS process isolation overhead for serverless-style workloads on Apple M1. Our central finding inverts the conventional FaaS performance narrative: *cold start is not the dominant isolation cost for sustained workloads on developer hardware*. Warm-request latency overhead (1.67–1.94×) exceeds cold-start overhead (1.24–1.37×) at every payload size. Throughput overhead reaches 3.74× at 1 MB.

We model this as a two-component isolation tax: a fixed per-request virtual network-stack traversal cost and a variable per-byte buffer copy cost. The fixed component dominates for small payloads; the per-byte component becomes the binding constraint at 100 KB and above. This model provides a practical decision framework for developers choosing between container and process isolation for local FaaS simulation, CI pipeline design, and on-device AI inference serving.

As ARM developer hardware becomes ubiquitous and containerised AI workloads proliferate, characterising the cost structure of container isolation on these platforms is increasingly important. Future work will extend this characterisation to Linux ARM64 to isolate the Docker Desktop VM contribution, and to GPU-accelerated inference workloads where the per-byte copy cost may interact with host-to-device memory transfer overhead.

References

[1] J. Manner, M. Endreß, T. Heckel, and G. Wirtz, "Cold start influencing factors in function as a service," in *Proc. IEEE/ACM Int. Conf. on Service-Oriented Computing (ICSOC) Workshops*, Hangzhou, China, Nov. 2018, pp. 1–12.

[2] J. Scheuner and P. Leitner, "Function-as-a-service performance evaluation: A multivocal literature review," *J. Syst. Softw.*, vol. 170, p. 110 function 856, Dec. 2020.

[3] H. Shafiei, A. Khonsari, and P. Mousavi, "Serverless computing: A survey of opportunities, challenges, and applications," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–32, Jan. 2022.

[4] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, Philadelphia, PA, Mar. 2015, pp. 171–172.

[5] P. Weeratunge, R. Buyya, and A. Ramamohanarao, "Performance characterization of ARM-based cloud instances for high-performance computing workloads," in *Proc. IEEE/ACM Int. Symp. on Cluster, Cloud and Internet Computing (CCGrid)*, 2021, pp. 191–200.

[6] D. Romero and N. Vijaykumar, "Rethinking file mapping for persistent memory," in *Proc. USENIX FAST*, 2021.

[7] Docker Inc., "Docker networking overview," *Docker Documentation*, 2024. [Online]. Available: https://docs.docker.com/network/

[8] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX ATC*, Boston, MA, Jul. 2018, pp. 133–146.

[9] G. Gerganov et al., "llama.cpp: LLM inference in C/C++," GitHub Repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[10] Cloud Native Computing Foundation, "containerd: An industry-standard container runtime," *CNCF Project*, 2024. [Online]. Available: https://containerd.io